Description of the elements of web.xml file

There are too many elements in the web.xml file. Here is the illustration of some elements that are used in the above web.xml file. The elements are as follows:

<web-app> represents the whole application.

<servlet> is sub element of <web-app> and represents the servlet.

<servlet-name> is sub element of <servlet> represents the name of the servlet.

<servlet-class> is sub element of <servlet> represents the class of the servlet.

<servlet-mapping> is sub element of <web-app>. It is used to map the servlet.

<url-pattern> is sub element of <servlet-mapping>. This pattern is used at client side to invoke the servlet.

RequestDispatcher in Servlet

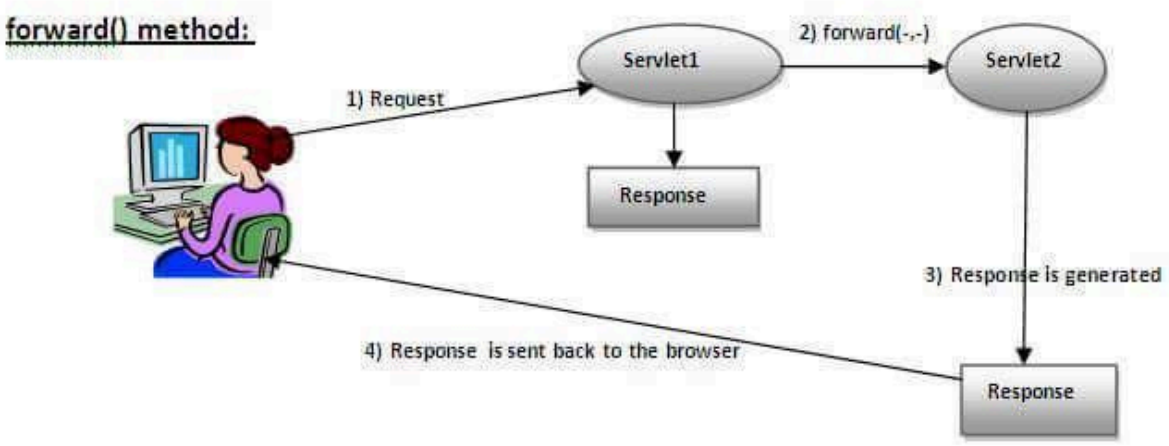
The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the ways of servlet collaboration.

There are two methods defined in the RequestDispatcher interface.

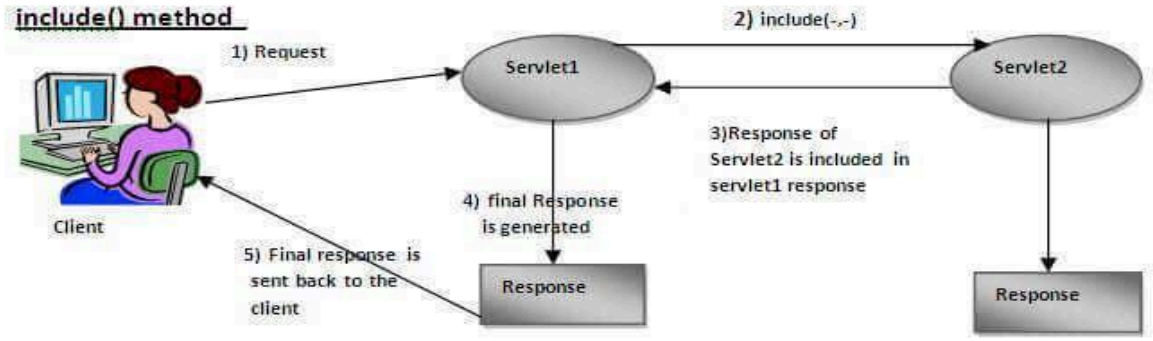
Methods of RequestDispatcher interface

The RequestDispatcher interface provides two methods. They are:

1. **public void forward(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:**Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.
2. **public void include(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:**Includes the content of a resource (servlet, JSP page, or HTML file) in the response.



As you see in the above figure, response of second servlet is sent to the client. Response of the first servlet is not displayed to the user.



As you can see in the above figure, response of second servlet is included in the response of the first servlet that is being sent to the client.

How to get the object of RequestDispatcher

The getRequestDispatcher() method of ServletRequest interface returns the object of RequestDispatcher. Syntax:

Syntax of getRequestDispatcher method

```
public RequestDispatcher getRequestDispatcher(String resource);
```

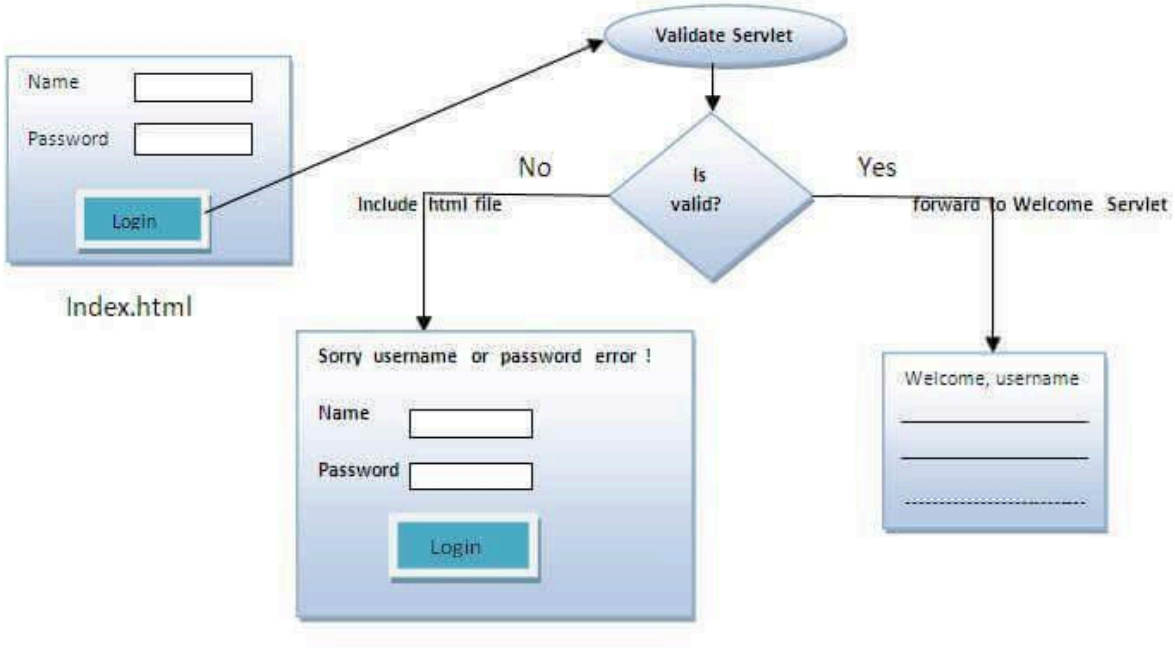
Example of using getRequestDispatcher method

```
RequestDispatcher rd=request.getRequestDispatcher("servlet2");

//servlet2 is the url-pattern of the second servlet

rd.forward(request, response);//method may be include or forward
```

Example of RequestDispatcher interface

In this example, we are validating the password entered by the user. If password is servlet, it will forward the request to the WelcomeServlet, otherwise will show an error message: sorry username or password error!. In this program, we are cheking for hardcoded information. But you can check it to the database also that we will see in the development chapter. In this example, we have created following files:

- **index.html file:** for getting input from the user.
- **Login.java file:** a servlet class for processing the response. If password is servet, it will forward the request to the welcome servlet.
- **WelcomeServlet.java file:** a servlet class for displaying the welcome message.
- **web.xml file:** a deployment descriptor file that contains the information about the servlet.



---

index.html

```
<form action="servlet1" method="post">

Name:<input type="text" name="userName"/><br/>

Password:<input type="password" name="userPass"/><br/>

<input type="submit" value="login"/>

</form>
```

Login.java

```
        import java.io.*;

        import javax.servlet.*;

        import javax.servlet.http.*;

public class Login extends HttpServlet {

public void doPost(HttpServletRequest request, HttpServletResponse response)

      throws ServletException, IOException {

    response.setContentType("text/html");

  PrintWriter out = response.getWriter();

    String n=request.getParameter("userName");

  String p=request.getParameter("userPass");

        if(p.equals("servlet"){

      RequestDispatcher rd=request.getRequestDispatcher("servlet2");

      rd.forward(request, response);

  }

  else{

      out.print("Sorry UserName or Password Error!");

      RequestDispatcher rd=request.getRequestDispatcher("/index.html");

      rd.include(request, response);

      }

  }
```

```
}
```

WelcomeServlet.java

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

 public class WelcomeServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)

      throws ServletException, IOException {

    response.setContentType("text/html");

   PrintWriter out = response.getWriter();

       String n=request.getParameter("userName");

   out.print("Welcome "+n);

   }

 }
```

---

web.xml

```
<web-app>

 <servlet>

   <servlet-name>Login</servlet-name>

   <servlet-class>Login</servlet-class>

 </servlet>

 <servlet>
```

```
  <servlet-name>WelcomeServlet</servlet-name>

  <servlet-class>WelcomeServlet</servlet-class>

</servlet>

  <servlet-mapping>

  <servlet-name>Login</servlet-name>

  <url-pattern>/servlet1</url-pattern>

</servlet-mapping>

<servlet-mapping>

  <servlet-name>WelcomeServlet</servlet-name>

  <url-pattern>/servlet2</url-pattern>

</servlet-mapping>



<welcome-file-list>

 <welcome-file>index.html</welcome-file>

</welcome-file-list>

          </web-app>
```

Difference between forward() and sendRedirect() method

forward() method	sendRedirect() method
The forward() method works at the server side.	The sendRedirect() method works at the client side.
It sends the same request and response objects to another servlet.	It always sends a new request.
It can work within the server only.	It can be used within and outside the server.
Example: request.getRequestDispacher("servlet2").forward(request,response);	Example: response.sendRedirect("servlet2");

Syntax of sendRedirect() method

```
public void sendRedirect(String URL)throws IOException;
```

Example of sendRedirect() method

```
response.sendRedirect("http://www.google.com");
```

---

example of sendRedirect method in servlet

In this example, we are redirecting the request to the google server. Notice that sendRedirect method works at client side, that is why we can our request to anywhere. We can send our request within and outside the server.

DemoServlet.java

```
import java.io.*;

import javax.servlet.*;

import javax.servlet.http.*;

 public class DemoServlet extends HttpServlet{

public void doGet(HttpServletRequest req,HttpServletResponse res)

throws ServletException,IOException

{

res.setContentType("text/html");

PrintWriter pw=res.getWriter();

 response.sendRedirect("http://www.google.com");

 pw.close();

}}
```

index.html

```
<!DOCTYPE html>

<html>

<head>

<meta charset="ISO-8859-1">

<title>sendRedirect example</title>
```

```
</head>

<body>

 <form action="MySearcher">

<input type="text" name="name">

    1.  <input type="submit" value="Google Search">

</form>

 </body>

</html>
```

MySearcher.java

```
import java.io.IOException;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

 public class MySearcher extends HttpServlet {

   protected void doGet(HttpServletRequest request, HttpServletResponse response)

          throws ServletException, IOException {

        String name=request.getParameter("name");

      response.sendRedirect("https://www.google.co.in/#q="+name);

   }

}
```

Attribute in Servlet

An **attribute in servlet** is an object that can be set, get or removed from one of the following scopes:

1. Request Scope: are most useful when processing the results of a submitted form. With request scope you are also guaranteed that no other request will be able to affect your objects in that request scope while your request is being handled. This scope has the shortest lifespan.
2. Session Scope: is associated with a user. When a user visits your web application, a session is created by the web container. So the session's lifespan lives as long as the user interacts with your application or when session.invalidate() is called.
3. Application Scope: or global scope is associated with your web application. This scope lives as long as the web application is deployed.

The servlet programmer can pass information from one servlet to another using attributes. It is just like passing object from one class to another so that we can reuse the same object again and again.

Attribute specific methods of ServletRequest, HttpSession and ServletContext interface

There are following 4 attribute specific methods. They are as follows:

1. **public void setAttribute(String name,Object object):**sets the given object in the application scope.
2. **public Object getAttribute(String name):**Returns the attribute for the specified name.
3. **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters as an Enumeration objects.
4. **public void removeAttribute(String name):**Removes the attribute with the given name from the servlet context.

Example of ServletContext to set and get attribute

In this example, we are setting the attribute in the application scope and getting that value from another servlet.

DemoServlet1.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class DemoServlet1 extends HttpServlet{
public void doGet(HttpServletRequest req,HttpServletResponse res)
{
try{

res.setContentType("text/html");
PrintWriter out=res.getWriter();

ServletContext context=getServletContext();
context.setAttribute("company","IBM");
out.println("Welcome to first servlet");
out.println("<a href='servlet2'>visit</a>");
out.close();
}
```

```
catch(Exception e){out.println(e);
}
 }
}
```

DemoServlet2.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class DemoServlet2 extends HttpServlet{
public void doGet(HttpServletRequest req,HttpServletResponse res)
{
try{
  res.setContentType("text/html");
PrintWriter out=res.getWriter();
ServletContext context=getServletContext();
String n=(String)context.getAttribute("company");
  out.println("Welcome to "+n);
out.close();
}
catch(Exception e){out.println(e);}
}
}
```

web.xml

```
<web-app>
  <servlet>
<servlet-name>s1</servlet-name>
<servlet-class>DemoServlet1</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>
  <servlet>
<servlet-name>s2</servlet-name>
<servlet-class>DemoServlet2</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>
  </web-app>
```

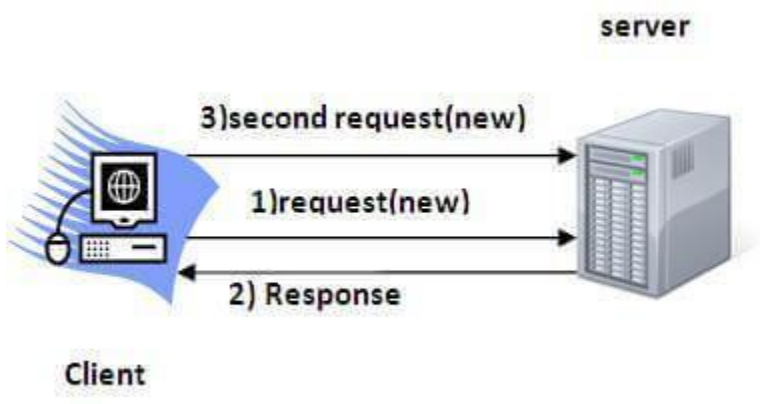
Session Tracking in Servlet

Session simply means a particular interval of time.

Session Tracking is a way to maintain state (data) of an user. It is also known as session management in servlet.

Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize to particular user.

HTTP is stateless that means each request is considered as the new request. It is shown in the figure given below:



Why use Session Tracking?

To recognize the user It is used to recognize the particular user.

Session Tracking Techniques

There are four techniques used in Session tracking:

1. **Cookies**
2. **Hidden Form Field**
3. **URL Rewriting**
4. **HttpSession**

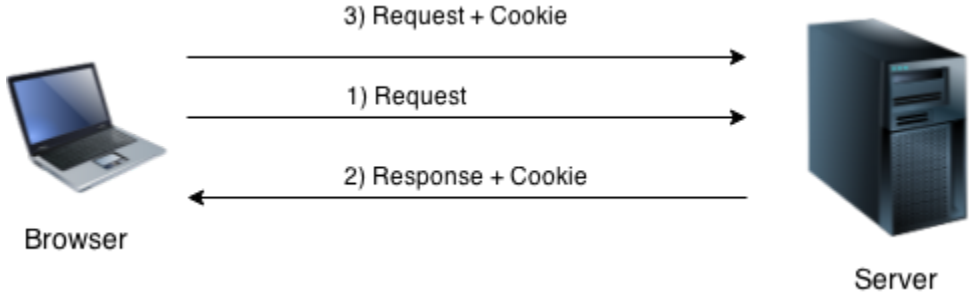
1.Cookies in Servlet

A **cookie** is a small piece of information that is persisted between the multiple client requests.

A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

How Cookie works

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.



Types of Cookie

There are 2 types of cookies in servlets.

1. Non-persistent cookie
2. Persistent cookie

Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie

It is **valid for multiple session** . It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if the cookie is disabled from the browser.
2. Only textual information can be set in the Cookie object.

Cookie class

javax.servlet.http.Cookie class provides the functionality of using cookies. It provides a lot of useful methods for cookies.

Useful Methods of Cookie class

Sr.No.	Method & Description
1	**public void setDomain(String pattern)** This method sets the domain to which cookie applies, for example google.com.
2	**public String getDomain()** This method gets the domain to which cookie applies, for example google.com.
3	**public void setMaxAge(int expiry)** This method sets how much time (in seconds) should elapse before the cookie expires. If you don't set this, the cookie will last only for the current session.
4	**public int getMaxAge()** This method returns the maximum age of the cookie, specified in seconds, By default, -1 indicating the cookie will persist until browser shutdown.
5	**public String getName()** This method returns the name of the cookie. The name cannot be changed after creation.
6	**public void setValue(String newValue)** This method sets the value associated with the cookie
7	**public String getValue()** This method gets the value associated with the cookie.
8	**public void setPath(String uri)** This method sets the path to which this cookie applies. If you don't specify a path, the cookie is returned for all URLs in the same directory as the current page as well as all subdirectories.
9	**public String getPath()** This method gets the path to which this cookie applies.
10	**public void setSecure(boolean flag)**

	This method sets the boolean value indicating whether the cookie should only be sent over encrypted (i.e. SSL) connections.
11	**public void setComment(String purpose)** This method specifies a comment that describes a cookie's purpose. The comment is useful if the browser presents the cookie to the user.
12	**public String getComment()** This method returns the comment describing the purpose of this cookie, or null if the cookie has no comment.

How to create Cookie?

Let's see the simple code to create cookie.

```
Cookie ck=new Cookie("user","sonoo jaiswal");//creating cookie object
response.addCookie(ck);//adding cookie in the response
```

How to delete Cookie?

Let's see the simple code to delete cookie. It is mainly used to logout or signout the user.

```
Cookie ck=new Cookie("user","");//deleting value of cookie
ck.setMaxAge(0);//changing the maximum age to 0 seconds
response.addCookie(ck);//adding cookie in the response
```

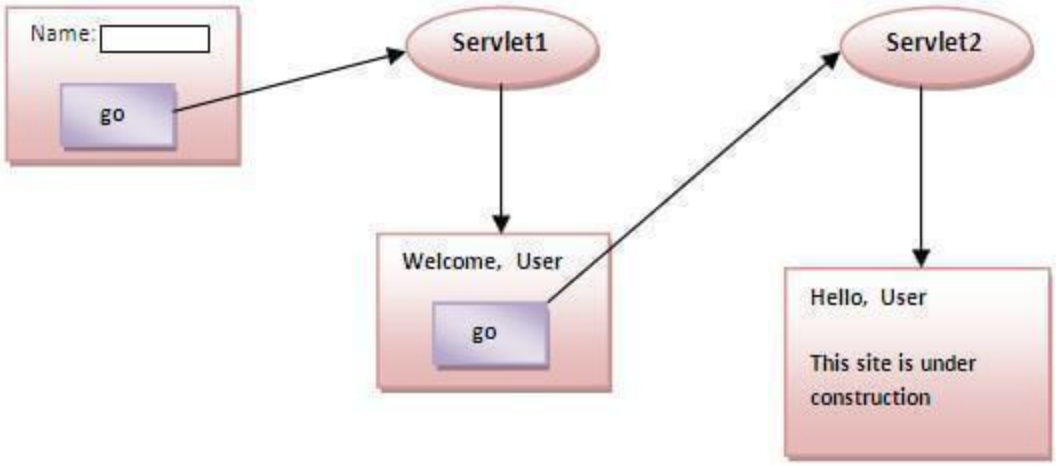
How to get Cookies?

Let's see the simple code to get all the cookies.

```
Cookie ck[]=request.getCookies();
for(int i=0;i<ck.length;i++){
 out.print("<br>"+ck[i].getName()+" "+ck[i].getValue());//printing name and value of cookie
}
```

Simple example of Servlet Cookies

In this example, we are storing the name of the user in the cookie object and accessing it in another servlet. As we know well that session corresponds to the particular user. So if you access it from too many browsers with different values, you will get the different value.



index.html

```
<form action="servlet1" method="post">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
 public class FirstServlet extends HttpServlet {
  public void doPost(HttpServletRequest request, HttpServletResponse response){
    try{
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String n=request.getParameter("userName");
    out.print("Welcome "+n);
    Cookie ck=new Cookie("uname",n);//creating cookie object
    response.addCookie(ck);//adding cookie in the response
    //creating submit button
    out.print("<form action='servlet2'>");
    out.print("<input type='submit' value='go'>");
    out.print("</form>");
    out.close();
      }catch(Exception e){System.out.println(e);}
  }
}
```

SecondServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SecondServlet extends HttpServlet {
```

```
public void doPost(HttpServletRequest request, HttpServletResponse response){
  try{

  response.setContentType("text/html");
  PrintWriter out = response.getWriter();
  Cookie ck[]=request.getCookies();
  out.print("Hello "+ck[0].getValue());
  out.close();
     }catch(Exception e){System.out.println(e);}
  }


}
```

web.xml

```
<web-app>
<servlet>
<servlet-name>s1</servlet-name>
<servlet-class>FirstServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>
<servlet>
<servlet-name>s2</servlet-name>
<servlet-class>SecondServlet</servlet-class>
</servlet>
<servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>
</web-app>
```

2) Hidden Form Field

In case of Hidden Form Field **a hidden (invisible) textfield** is used for maintaining the state of an user.

In such case, we store the information in the hidden field and get it from another servlet. This approach is better if we have to submit form in all the pages and we don't want to depend on the browser.

Let's see the code to store value in hidden field.

```
1.  <input type="hidden" name="uname" value="Vimal Jaiswal">
```

Here, uname is the hidden field name and Vimal Jaiswal is the hidden field value.

Real application of hidden form field

It is widely used in comment form of a website. In such case, we store page id or page name in the hidden field so that each page can be uniquely identified.

Advantage of Hidden Form Field

1. It will always work whether the cookie is disabled or not.

Disadvantage of Hidden Form Field:

1. It is maintained at server side.
2. Extra form submission is required on each pages.
3. Only textual information can be used.

Example of using Hidden Form Field

In this example, we are storing the name of the user in a hidden textfield and getting that value from another servlet.

index.html

```
<form action="servlet1">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
  public class FirstServlet extends HttpServlet {
public void doGet(HttpServletRequest request, HttpServletResponse response){
      try{
        response.setContentType("text/html");
      PrintWriter out = response.getWriter();
         String n=request.getParameter("userName");
```

```
        out.print("Welcome "+n);
            //creating form that have invisible textfield
        out.print("<form action='servlet2'>");
        out.print("<input type='hidden' name='uname' value='"+n+"'>");
        out.print("<input type='submit' value='go'>");
        out.print("</form>");
        out.close();
            }catch(Exception e){System.out.println(e);}
    }
  }
```

SecondServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class SecondServlet extends HttpServlet {
public void doGet(HttpServletRequest request, HttpServletResponse response)
    try{
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    //Getting the value from the hidden field
    String n=request.getParameter("uname");
    out.print("Hello "+n);

    out.close();
        }catch(Exception e){System.out.println(e);}
  }
}
```

web.xml

```
<web-app>
 <servlet>
<servlet-name>s1</servlet-name>
```

```
<servlet-class>FirstServlet</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>
 <servlet>
<servlet-name>s2</servlet-name>
<servlet-class>SecondServlet</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>
  </web-app>
```

3)URL Rewriting

In URL rewriting, we append a token or identifier to the URL of the next Servlet or the next resource. We can send parameter name/value pairs using the following format:

url?name1=value1&name2=value2&??

A name and a value is separated using an equal = sign, a parameter name/value pair is separated from another parameter using the ampersand(&). When the user clicks the hyperlink, the parameter name/value pairs will be passed to the server. From a Servlet, we can use getParameter() method to obtain a parameter value.

Advantage of URL Rewriting

1. It will always work whether the cookie is disabled or not (browser independent).
2. Extra form submission is not required on each pages.

Disadvantage of URL Rewriting

1. It will work only with links.
2. It can send Only textual information.

Example of using URL Rewriting

In this example, we are maintaning the state of the user using link. For this purpose, we are appending the name of the user in the query string and getting the value from the query string in another page.

index.html

```
<form action="servlet1">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;



public class FirstServlet extends HttpServlet {

public void doGet(HttpServletRequest request, HttpServletResponse response){
    try{

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
        String n=request.getParameter("userName");
    out.print("Welcome "+n);
      //appending the username in the query string
    out.print("<a href='servlet2?uname="+n+"'>visit</a>");
    out.close();
        }catch(Exception e){System.out.println(e);}
  }
 }
```

SecondServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
 public class SecondServlet extends HttpServlet {
 public void doGet(HttpServletRequest request, HttpServletResponse response)
     try{

     response.setContentType("text/html");
     PrintWriter out = response.getWriter();

     //getting value from the query string
     String n=request.getParameter("uname");
     out.print("Hello "+n);

     out.close();

         }catch(Exception e){System.out.println(e);}
   }



}
```

web.xml

```
<web-app>
  <servlet>
<servlet-name>s1</servlet-name>
<servlet-class>FirstServlet</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>
  <servlet>
```

```
<servlet-name>s2</servlet-name>
<servlet-class>SecondServlet</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>
</web-app>
```

4) HttpSession interface

In such case, container creates a session id for each user.The container uses this id to identify the particular user.An object of HttpSession can be used to perform two tasks:

1. bind objects
2. view and manipulate information about a session, such as the session identifier, creation time, and last accessed time.

How to get the HttpSession object ?

The HttpServletRequest interface provides two methods to get the object of HttpSession:

1. **public HttpSession getSession():**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **public HttpSession getSession(boolean create):**Returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session.

Commonly used methods of HttpSession interface

1. **public String getId():**Returns a string containing the unique identifier value.
2. **public long getCreationTime():**Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
3. **public long getLastAccessedTime():**Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.

4. **public void invalidate():**Invalidates this session then unbinds any objects bound to it.

Advantage of HttpSession :

1. HttpSession provides secure, server-side session management
2. easy API available
3. Persistent data across requests, custom attributes, configurable timeouts, and support for clustered environments.

Example of using HttpSession

In this example, we are setting the attribute in the session scope in one servlet and getting that value from the session scope in another servlet. To set the attribute in the session scope, we have used the setAttribute() method of HttpSession interface and to get the attribute, we have used the getAttribute method.

index.html

```
<form action="servlet1">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
```

FirstServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
 public class FirstServlet extends HttpServlet {
 public void doGet(HttpServletRequest request, HttpServletResponse response){
      try{
        response.setContentType("text/html");
      PrintWriter out = response.getWriter();
            String n=request.getParameter("userName");
      out.print("Welcome "+n);
           HttpSession session=request.getSession();
```

```
        session.setAttribute("uname",n);
          out.print("<a href='servlet2'>visit</a>");
                out.close();
                }catch(Exception e){System.out.println(e);}
    }
  }
```

SecondServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SecondServlet extends HttpServlet {
  public void doGet(HttpServletRequest request, HttpServletResponse response)
        try{
          response.setContentType("text/html");
        PrintWriter out = response.getWriter();
                HttpSession session=request.getSession(false);
        String n=(String)session.getAttribute("uname");
       out.print("Hello "+n);
          out.close();
                }catch(Exception e){System.out.println(e);}
    }
  }
```

web.xml

```
<web-app>
  <servlet>
<servlet-name>s1</servlet-name>
<servlet-class>FirstServlet</servlet-class>
</servlet>
  <servlet-mapping>
<servlet-name>s1</servlet-name>
```

```
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>
<servlet>
<servlet-name>s2</servlet-name>
<servlet-class>SecondServlet</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>
 </web-app>
```